

TASKORA

Team Management Application

SOFTWARE REQUIREMENTS DOCUMENT

Prepared by:

Raynald (Programmer) | Duard (System Analyst)
Yovela (Product Owner) | Nicholas (Tech Lead)

1. Introduction

1.1 Purpose of this Document

This Software Requirements Document (SRD) defines the complete functional and non-functional requirements for Taskora, a web-based team management application. It serves as the authoritative technical reference for all development, design, and deployment activities, and is intended for use by developers, system analysts, product owners, stakeholders, and academic evaluators.

1.2 Project Overview

Taskora is a collaborative task management platform designed to help teams organize their work, assign responsibilities, track individual contributions, and meet deadlines efficiently. The application targets student groups, startup teams, and small-to-medium professional teams that suffer from coordination overhead and lack of accountability in distributed workflows.

2. Background

Group projects often become inefficient not because members lack ability, but because coordination is poorly managed. Teams typically rely on multiple separate platforms such as, chat applications, shared documents, and personal reminders, which leads to unclear responsibilities, missed deadlines, and uneven workload distribution. As a result, some members contribute significantly more than others with no reliable record to objectively evaluate participation.

2.1 Problem Statement

Modern collaborative teams, especially in academic and early-stage startup environments, face systemic coordination failures. These failures are not attributable to individual incompetence but arise from structural gaps in tooling and process. The primary problems identified are:

- Unclear task distribution and responsibilities
- Missed deadlines due to lack of centralized tracking
- Uneven workload distribution among members
- No reliable system to objectively measure individual contributions
- Difficulty accessing shared project resources
- Limited visibility for leaders to monitor overall progress.

3. Proposed Solution

Taskora addresses these issues by providing a centralized workspace where tasks, deadlines, contributions, and shared resources are managed in one system. Features such as task assignment, contribution tracking, calendars, and resource shortcuts make responsibilities clear and visible to all members. By organizing collaboration and automatically recording activity, the application improves accountability, reduces miscommunication, and allows fair evaluation of each member's contribution.

3.1 Core Features

- Centralized task management
- Task assignment with priority and status tracking
- Calendar with deadlines (end date)

- Contribution tracking system
- Task verification system (verified by leader)
- Shared resource shortcuts (Google Drive, Docs, Canva, etc.)
- Member management & role-based access
- Comment section on each task and optional team chat / forum
- Authentication & Authorization
- Real-time in-app notifications via Socket.io
- Activity log with timestamped workspace history

4. Requirements Gathering

4.1 Methodology

Requirement gathering was conducted using online questionnaires (Google Forms) distributed to university students who frequently participate in group projects. This approach enables efficient collection from a large number of students (the target users of Taskora). Anonymous responses encourage honest feedback, and the method is time-efficient for a student project with limited resources.

4.2 Respondents Profile

A total of 22 responses were collected between March 2–10, 2026. Respondents came from a wide range of majors, indicating broad cross-disciplinary applicability. The overwhelming majority (20 out of 22) are 2nd-year students, the stage at which university group projects are most frequent and intensive.

4.3 Validated Problems Data

Problem Statement	Yes / Often	Response Rate
Do you experience unclear task distribution?	13 / 22	59.1%
Have you experienced uneven workload distribution?	19 / 22	86.4%
Is it difficult to objectively evaluate contributions?	13 / 22	59.1%

4.3 Challenges Identified

Challenge	Mentions	% of Respondents
Unresponsive / inactive team members	~9	40.9%
Unclear / uneven task distribution	~7	31.8%
Communication difficulties	~4	18.2%
Miscellaneous (language, scheduling, teammate selection)	~2	9.1%

The most prominent pain point is unresponsive or non-contributing team members. This is closely followed by ambiguity in task ownership, many teams lack a clear system for distributing and monitoring work. Communication gaps complete the top three issues, reinforcing the need for a centralized, structured workspace.

5. User of the System

Taskora identifies two primary user roles. All four team members are expected to contribute to development. The Product Owner and System Analyst roles are organizational responsibilities and do not exempt members from coding duties given the team size. Both are filled by university students; the distinction is organizational rather than demographic.

Role	Description	Typical Responsibilities
Team Leader	A student assigned or elected as the coordinator of a group project.	Create workspace, assign tasks, set deadlines, verify completions, manage members, monitor overall progress.
Team Member	A student who is part of a project team and carries out assigned tasks.	View assigned tasks, update task status, communicate via comments, access shared resources.

6. Key Features & Priority

Respondents rated the importance of each proposed feature on a scale of 1–5 (1 = Not important, 5 = Very important).

#	Feature	Avg.	Priority	Source
1	Centralised Task Management System	4.09	High	Survey
2	Task Assignment to Specific Members	3.95	High	Survey
3	Task Priority Levels (High / Medium / Low)	3.95	High	Survey
4	Task Status Tracking (To Do / In Progress / Done)	3.64	Medium	Survey
5	Shared Calendar with Start and End Dates	3.77	Medium	Survey
6	Automatic Deadline Reminders / Notifications	3.95	High	Survey
7	Task Verification by Leader before Marking Done	3.86	Medium	Survey
8	Comment Section on Each Task	3.50	Medium	Survey
9	Shared Resource Shortcuts (Drive, Docs, Canva)	3.86	Medium	Survey
10	Role-Based Access (Leader & Member)	3.59	Medium	Survey
11	Built-in Team Chat / Forum	3.41	Medium	Survey
12	Activity Log / History (who did what, when)	—	High	User Input
13	Contribution Summary at Project Completion	—	High	User Input

7. Functional Requirements

Functional requirements define what the system shall do. They are derived from the validated features and survey findings, organized by module.

7.1 Workspace & Project Management

The system shall allow a Team Leader to create a project workspace and invite members via link or email. A dashboard shall give all members a clear overview of project status, task progress, and team activity.

7.2 Task Management

The system shall allow the Team Leader to create tasks with a title, assignee, priority level (High / Medium / Low), start date, and deadline. Members shall update task status across three stages: To Do, In Progress, and Done. Tasks shall require Team Leader verification before being marked complete. The Team Leader shall be able to edit or delete tasks as needed.

7.3 Deadline & Calendar

The system shall provide a shared calendar displaying all task deadlines for the workspace. In-app reminder notifications shall be delivered in real time via Socket.io to the assigned member and Team Leader as deadlines approach, and overdue tasks shall be visually flagged.

7.4 Contribution Tracking

The system shall automatically record each member's completed and verified tasks. Contribution is measured by the total number of verified tasks completed by each member. This metric reflects task quantity only. Task complexity and time spent are not currently weighted. This is a known limitation and may be addressed in a post-MVP iteration. A contribution summary showing each member's task count and percentage share shall be visible to all members at any time, with a final breakdown generated at project completion.

7.5 Activity Log

The system shall maintain a timestamped log of key workspace events, including task creation, status changes, and member activity. Each entry shall show the acting user, action type, and timestamp. The log shall be read-only and visible to all members.

7.6 Communication

The system shall provide a comment thread on each task for context-specific discussion. An optional team-level chat or forum shall be available for general project communication.

7.7 Shared Resource Shortcuts

The system shall allow members to save named links to external resources such as Google Drive, Google Docs, and Canva. All shortcuts shall be displayed in a dedicated panel within the workspace.

7.8 Notification

The system shall deliver in-app real-time notifications via Socket.io for seven trigger types: workspace invitation, task assignment, comment posted, comment replied, deadline reminder, task verified, and status changed. Notifications shall be stored in the database with an isRead flag, and users shall be able to mark individual notifications or all notifications as read. The notification bell icon shall display an unread count badge and update without page refresh.

7.9 Member Management

Team Leaders may invite users to a workspace via a signed JWT invite link or by email with an in-app notification (no SMTP). Members may leave a workspace at any time via a Leave Workspace option with a confirmation dialog; if the leaving member is the sole Team Leader, they must transfer ownership before leaving. Team Leaders may remove members from the workspace.

8. Non-Functional Requirements

Non-functional requirements define how the system shall perform and constrain overall system quality. They are derived from the target user profile and the operational context of a university student environment.

Category	Requirement
Usability	The interface shall be clean and intuitive, allowing a first-time user to create a workspace and assign a task without needing a tutorial. The design shall avoid unnecessary complexity.
Performance	Pages shall load within 3 seconds under normal network conditions. In-app deadline reminder notifications shall be delivered in real time via Socket.io within 5 seconds of the scheduled trigger time.
Reliability	The system shall maintain at least 99% uptime during active semester periods, excluding scheduled maintenance windows.
Security	User passwords shall be stored using industry-standard hashing. All data in transit shall be encrypted via HTTPS. Role-based access control shall be enforced server-side.
Scalability	The system shall support up to 20 members per workspace without performance degradation.
Accessibility	The system shall run on all modern browsers without installation and shall be responsive on mobile screen sizes.

9. Feature List Summary

Priority Tier	Features
High — Core MVP	Project workspace creation & member management; Task creation with assignee, priority, start date & deadline; Task status tracking (To Do / In Progress / Done); Task verification workflow (leader approval); Automatic deadline reminders; Contribution tracking & final summary; Activity log with timestamped history; Real-time in-app notifications (7 trigger types)
Medium — Extended MVP	Shared project calendar with deadline visualization; Comment threads on individual tasks; Project dashboard with progress overview; Activity log with colour-coded dot indicators and inline workspace filter; Cross-workspace task aggregation on Assigned Task page
Low / P3 — Delivered	Shared resource shortcuts panel (Google Drive, Docs, Canva, etc.); Full production deployment to Vercel + Railway/Render with Vercel Cron
Post-MVP (Not in Scope)	Advanced analytics and reporting; Integration with external calendar apps (Google Calendar, etc.); Team-level chat or forum

10. UI/UX Design

The main design priority is User Experience (UX). Since the system targets students managing group projects, the application must be easy to understand, simple to navigate, and efficient to use. A good UX ensures users can quickly assign tasks, monitor deadlines, track contributions, and access shared resources without confusion. While UX leads, User Interface (UI) is equally important — a clean, organized interface helps users understand and interact with features intuitively.

10.1 Figma Prototype

The interface design and prototype were created using Figma:

<https://embed.figma.com/design/Q24InnJxAqzhFvIFKsQITr/Untitled?node-id=0-1&t=g5gwm4fUc6krZypH-1&embed-host=notion&footer=false&theme=system>

10.2 Light & Dark Mode

Full light and dark mode support was introduced in Sprint 2 in response to Sprint 1 stakeholder feedback requesting improved visual variety and theme options. Overall UI polish was iteratively improved each sprint, with Sprint 4 delivering the highest stakeholder satisfaction score of 4.84/5.

11. System Architecture

Component	Technology	Rationale
Arch. Pattern	Layered Architecture	Separates the backend into distinct horizontal layers: Routes → Controllers → Services → Repositories → Models. Each layer has a single responsibility — business logic lives in Services, keeping Controllers thin. Makes each layer independently testable and allows team members to work in parallel.
Frontend	ReactTS + Vite	Component-based library for dynamic, responsive UIs (dashboards, status updates, audit logs). Vite provides fast HMR for iterative development and optimized production builds.
Backend	TypeScript + Express.js + Socket.io	TypeScript adds static typing, reducing runtime errors. Express.js is a well-documented Node.js framework that gives the team full control over server logic and API design. Socket.io enables real-time, bidirectional communication for instant task updates and live notifications without page refreshes.
Database	PostgreSQL (Neon)	Relational DBMS suited for structured data (users, projects, tasks, deadlines). Ensures data integrity through relational constraints and supports efficient querying for task assignment and dashboards.
Deployment — API	Vercel (serverless)	REST API deployed to Vercel with all environment variables configured in dashboard. Cold-start response under 3 seconds. Vercel Cron registered for /api/cron/overdue (hourly) and /api/cron/reminders.
Deployment — Socket.io	Railway / Render (persistent)	Persistent server required for Socket.io (stateful WebSocket connections). UptimeRobot pings every ≤14 minutes to prevent cold-start sleep delays.
Deployment — Frontend	Vercel	Frontend deployed to Vercel production URL. VITE_API_URL and VITE_SOCKET_URL point to production endpoints. All pages load without console errors; end-to-end flows verified in production.

12. Process Model (SDLC — Agile/Scrum)

The Agile/Scrum model was adopted to allow iterative development and continuous improvement. Development was divided into sprints, with a demo to stakeholders and periodic testing conducted every two weeks. This enabled the team to incorporate feedback early, adapt to changes, and deliver working features incrementally.

Sprint	Dates	Focus	Outcome
Sprint 0	Before 27 Apr	Requirements, SRD, Product Backlog, UML, Figma prototype	All pre-development artifacts baselined
Sprint 1	27 Apr – 3 May	Authentication, workspace CRUD, core frontend pages	Core auth and workspace flows live; 4.6/5 satisfaction

Sprint	Dates	Focus	Outcome
Sprint 2	4 – 12 May	Invite/join flow, full task CRUD, activity log, light/dark mode	Task management fully operational; 3.8/5 satisfaction
Sprint 3	12 – 19 May	Real-time (Socket.io), notifications, comments, calendar, contribution	Full real-time stack and collaboration features delivered; 4.5/5 satisfaction
Sprint 4	20 – 27 May	Resource Shortcuts, production deployment, quality fixes, handover	System fully deployed to production; all defects resolved; 4.84/5 satisfaction

13. System Constraints

Constraints are conditions the system must operate within, regardless of design decisions. They are non-negotiable limits imposed by the environment, team, budget, or technology.

Category	Constraint
Deployment	The REST API must be deployable on Vercel's serverless platform. Vercel does not support persistent WebSocket connections, so Socket.io must be deployed separately on Railway or Render.
Budget	The project must be deployable within the free tier of Vercel, Neon PostgreSQL, and Railway/Render. No paid cloud infrastructure is in scope.
Time	The full system must be designed, developed, tested, and deployed within one academic semester. Development is divided into 4 one-week sprints (Sprint 1–4), preceded by Sprint 0 (preparation).
Team Size	The development team consists of exactly 4 members: 1 Tech Lead / Frontend developer, 1 Backend programmer, 1 Product Owner, and 1 System Analyst. All members are expected to contribute to coding.
Communication	No SMTP (email) integration is permitted. All user notifications must be delivered in-app via Socket.io only.
Database	The system must use a relational database (PostgreSQL via Neon). No NoSQL or other database types are permitted in v1.0.
Language / Stack	Backend must be written in TypeScript using Express.js. Frontend must use React with Vite. The ORM must be Prisma. These are fixed technology choices and are not negotiable for this version.
Workspace Size	The system is designed for workspaces of up to 20 members. Behaviour beyond this limit is undefined and not required to be supported.